Step 13 — AI agents (MCP) and developer docs

What it does, and how to test it on your Docker stack step by step with the commands

Step	Phase 3, step 13: MCP endpoint for AI agents, OpenAPI 3.1 description, docs page for developers
Status	Built 2026-10-26; not yet tested by the owner
Branch	`claude/cloud-vs-local-7xi988` (via a pull request), folder `Owis_Find_Deal_Engine/server`
New endpoints	`POST /mcp` (API keys) · `GET /api/v1/openapi.json` · `GET /docs/` (public)
Settings	none new; nothing to add to .env
Database	no change
Date	26 October 2026

> **How to read this guide.** Part 1 explains what step 13 does. Part 2 is the test: steps 0–10 with the commands to type (Git Bash on Windows) and what you should see. Every step uses the cached `usb c hub` / `jor` search or no search at all; only if that search is no longer in the cache (older than 48 h) does step 4 run one live search.

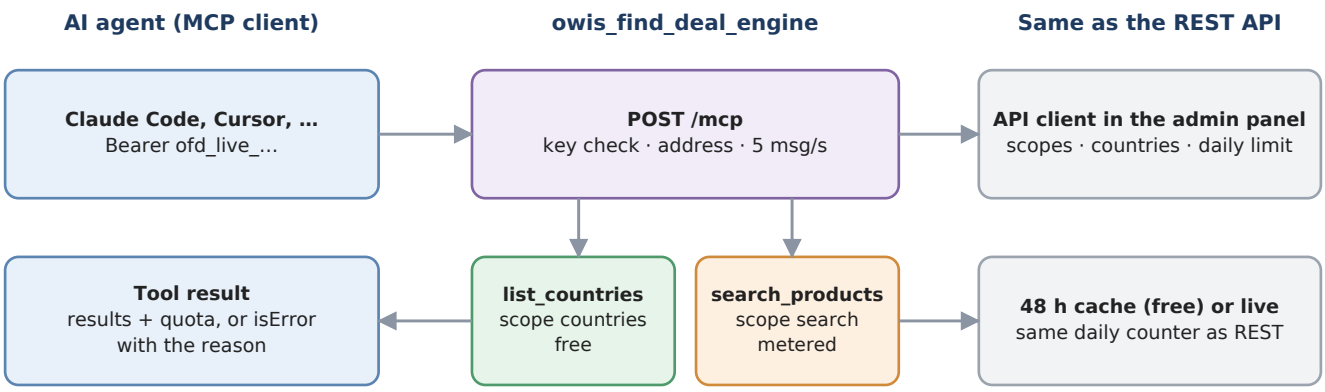
Part 1 — What step 13 does

Step 13 lets AI agents use the search directly, and gives developers the description and documentation they need to build on it.

1. MCP endpoint for AI agents

The **Model Context Protocol** (MCP) is the standard way AI agents (Claude, Cursor, VS Code, your own agents) discover and call tools. The service now has an MCP endpoint at `POST /mcp`. An agent connects with the **same API key** as the REST API and sees two tools:

Tool	Needs scope	What it does
`list_countries`	`countries`	The countries this key may search in, with currency and marketplaces. Free.
`search_products`	`search`	Searches one country for a product (`title`, `country`, optional `refresh`). Same results as `/api/v1/search`, plus the allowance left. Cached answers free, a live search counts once.



An agent calls the tools with its API key; the key's scopes, countries and daily limit apply as on the REST API.

- **API keys only.** A user's login token is refused (401): agents are servers, not app users.
- **Problems come back to the agent as readable tool errors** (`isError`), e.g. `quota_exceeded` with the seconds to wait, `country_not_allowed`, or a missing scope, so the agent can explain or adjust instead of failing.
- **The same limits as the REST API:** the key's daily allowance (one counter for REST and MCP), its searches per second, and at most 5 MCP messages per second.
- **Both protocol generations are spoken:** the new stateless version 2026-07-28 and the older 2025-03-26 … 2025-11-25 (with a start handshake). Tested with the official TypeScript and Go MCP libraries.
- **A new search takes 20–60 s**: the tool waits for it, so agents need a timeout of about 90 s.

2. Developer docs and OpenAPI

- `http://localhost:3002/docs/` — one page for developers: quick start, keys, search, asynchronous search, webhooks (with code to check the signature in Python and Node.js), how to connect MCP clients, limits and all error codes. Public, no scripts.
- `http://localhost:3002/api/v1/openapi.json` — the machine-readable description (OpenAPI 3.1) of every public endpoint, including the webhook. Developers load it into Swagger UI, Postman or Insomnia, or generate client code from it.

Part 2 — Test steps

The commands are for **Git Bash** on Windows, in the `Owis_Find_Deal_Engine/server` folder. In PowerShell, type `curl.exe` instead of `curl`. Replace every <…> with your own value. The Git Bash tips from the step 12 guide apply (`export NAME=value` without spaces, variables live in one window only).

Step 0 — Update and start

```
cd claude-ai-clould-repo
git checkout claude/cloud-vs-local-7xi988
git pull
cd Owis_Find_Deal_Engine/server
docker compose up -d --build
docker compose ps                      # all four containers "healthy" after ~30 s
curl localhost:3002/api/v1/health      # {"status":"ok"}
```

Until the step 13 pull request is merged, use `git checkout claude/optimistic-brown-tqclh3` instead (the branch with step 13).

✓ **Expected:** the four containers are healthy; nothing to add to .env, no database change.

Step 1 — The docs page and the OpenAPI file (free)

Open `http://localhost:3002/docs/` in your browser. Then:

```
curl -s localhost:3002/api/v1/openapi.json | head -c 300; echo
curl -s -o /dev/null -w "%{http_code}\n" localhost:3002/docs     # 301 -> /docs/
```

✓ **Expected:** the page shows Owis Find Deal API with a menu (Quick start … OpenAPI) and follows your light / dark setting; the JSON begins with `"openapi": "3.1.0"`. Optional: paste the JSON into `https://editor.swagger.io` (File → Import) to browse it.

Step 2 — Make a client for the agent

Admin panel → API clients → New client: Name `step13 agent`, Live searches per day `20`, scopes **search** and **countries** both ticked, country **jor** → **Create client and key** → **Copy** → **I have saved it**. Then:

```
export KEY=ofd_live_<...>
echo "${KEY:0:12}"        # ofd_live_xxx
```

The step 12 client also works if it has both scopes.

Step 3 — The handshake and the tool list (free)

```
curl -s localhost:3002/mcp -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" -H "MCP-Protocol-Version: 2025-06-18" \
  -d '{"jsonrpc":"2.0","id":1,"method":"initialize",
       "params":{"protocolVersion":"2025-06-18","capabilities":{},"clientInfo":{"name":"gitbash","version":"1"}}}'

curl -s localhost:3002/mcp -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" -H "MCP-Protocol-Version: 2025-06-18" \
  -d '{"jsonrpc":"2.0","id":2,"method":"tools/list",
       "params":{}}'
```

✓ **Expected:** the first answer has `"protocolVersion":"2025-06-18"`, `"serverInfo":{"name":"owis_find_deal_engine"` and `"instructions"`; the second lists two tools, `list_countries` and `search_products`, each with an inputSchema and outputSchema.

Step 4 — Call the tools

```
curl -s localhost:3002/mcp -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" -H "MCP-Protocol-Version: 2025-06-18" \
  -d '{"jsonrpc":"2.0","id":3,"method":"tools/call",
       "params":{"name":"list_countries","arguments":{}}}'

curl -s localhost:3002/mcp -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" -H "MCP-Protocol-Version: 2025-06-18" \
  -d '{"jsonrpc":"2.0","id":4,"method":"tools/call",
       "params":{"name":"search_products","arguments":{"title":"usb c hub","country":"jor"}}}'
```

✓ **Expected:** list_countries shows only **Jordan** (the key is limited to jor). search_products answers with `"structuredContent":{"query":"usb c hub",…,"results":[…]` and `"quota":{"plan":"api","limit":20,…}`. If the search was cached (step 12 test less than 48 h ago, or run it twice) `"cached":true` and the remaining count does not change; otherwise the first call takes 30–50 s and counts once.

Step 5 — Errors the agent can read (free)

```
curl -s localhost:3002/mcp -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" -H "MCP-Protocol-Version: 2025-06-18" \
  -d '{"jsonrpc":"2.0","id":5,"method":"tools/call",
       "params":{"name":"search_products","arguments":{"title":"usb c hub","country":"usa"}}}'

curl -s localhost:3002/mcp -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" -H "MCP-Protocol-Version: 2025-06-18" \
  -d '{"jsonrpc":"2.0","id":6,"method":"tools/call",
       "params":{"name":"search_products","arguments":{"title":"x","country":"jor"}}}'

curl -s localhost:3002/mcp -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" -H "MCP-Protocol-Version: 2025-06-18" \
  -d '{"jsonrpc":"2.0","id":7,"method":"tools/call",
       "params":{"name":"buy_everything","arguments":{}}}'
```

✓ **Expected:** usa → a normal answer with `"isError":true` and the text `country_not_allowed`; title x → `isError` with `invalid_request` (title must be 2-200 characters); the made-up tool → a JSON-RPC `"error":{"code":-32602` (unknown tool).

Step 6 — The new protocol version 2026-07-28 (free)

The newest MCP version has no handshake: each request names its version in the body and in three headers, which must match.

```
curl -s localhost:3002/mcp -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" \
  -H "MCP-Protocol-Version: 2026-07-28" -H "Mcp-Method: server/discover" \
  -d '{"jsonrpc":"2.0","id":8,"method":"server/discover","params":{"_meta":{
       "io.modelcontextprotocol/protocolVersion":"2026-07-28",
       "io.modelcontextprotocol/clientCapabilities":{}}}}'

# the same with a wrong header -> 400
curl -s -w "\n%{http_code}\n" localhost:3002/mcp -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" \
  -H "MCP-Protocol-Version: 2026-07-28" -H "Mcp-Method: tools/list" \
  -d '{"jsonrpc":"2.0","id":9,"method":"server/discover","params":{"_meta":{
       "io.modelcontextprotocol/protocolVersion":"2026-07-28",
       "io.modelcontextprotocol/clientCapabilities":{}}}}'
```

✓ **Expected:** the first lists `"supportedVersions":["2026-07-28","2025-11-25","2025-06-18","2025-03-26"]` with `"resultType":"complete"`; the second answers `400` with `"code":-32020` (Header mismatch).

Step 7 — Who is refused (free)

```
# no key, and a login token: both 401
curl -s -X POST localhost:3002/mcp -d '{}'; echo
curl -s -X POST localhost:3002/mcp -H "Authorization: Bearer $ADMIN_TOKEN" -d '{}'; echo
# GET is not used by this server: 405
curl -s -o /dev/null -w "%{http_code}\n" localhost:3002/mcp
```

✓ **Expected:** `missing bearer token`; `this endpoint takes an API key …, not a login token` (with a fresh admin token from step 1 of the step 12 guide; an expired one gives `invalid or expired token`, also fine); `405`.

A key without the search scope: make a client `step13 lister` with only the scope **countries**, export its key as LKEY, and call search_products with it (step 4, $LKEY instead of $KEY).

✓ **Expected:** `isError` with `forbidden`: this API key lacks the scope "search" needed for search_products; list_countries works with it.

Step 8 — Connect a real agent (optional, free with the cached search)

Claude Code (if installed):

```
claude mcp add --transport http owis-find-deal http://localhost:3002/mcp \
  --header "Authorization: Bearer $KEY"
claude mcp list            # owis-find-deal: ... - ✓ Connected
claude                     # then ask: "Use owis-find-deal to find a usb c hub in Jordan"
```

Or the MCP Inspector (a test tool in the browser; needs Node.js):

```
npx @modelcontextprotocol/inspector
```

In the page that opens: Transport **Streamable HTTP**, URL `http://localhost:3002/mcp`, under Authentication add the header `Authorization` = `Bearer ofd_live_…` → **Connect** → **Tools** → **List Tools** → run `search_products` with `usb c hub` / `jor`.

✓ **Expected:** the agent (or Inspector) lists both tools and shows the offers per marketplace. Remove the Claude Code entry afterwards with `claude mcp remove owis-find-deal` if you like.

Step 9 — Dashboard and logs (free)

```
docker compose logs server | grep '"/mcp"' | tail -3
docker compose logs server | grep -c ofd_live_      # 0
```

✓ **Expected:** one log line per MCP request with `"user":"client:<id>"`; the key never appears in the log. After ~30 s the dashboard's **API clients** table shows step13 agent with its cached (and, if any, live) searches.

Step 10 (optional) — Daily limit through MCP (costs one live search)

Make a client with Live searches per day **1**, run search_products once with a **new** title, then again with another new title.

✓ **Expected:** the second call answers `isError` with `quota_exceeded` and `retry_after_seconds` (the seconds until midnight UTC); no second live search runs.

What to report back

1. Which steps passed or failed, with the answer and `docker compose logs server` lines for any failure.
2. Which agents you plan to connect (Claude Code, Cursor, your own code), and whether 90 s for a new search is acceptable for them, or they should use the asynchronous search instead (a third tool, e.g. `start_search` + `get_search`, can be added).
3. Whether the docs page is clear for an outside developer: wording, missing examples, order.
4. Whether the docs page and OpenAPI file should stay public in production (now: yes, they hold no secrets), or be behind a login.